

CoderAxo AI/ML Internship Program — 2026

# PROJECT REPORT

## Real-Time Driver Drowsiness Detection System

### DrowsyGuard

|                 |                                            |
|-----------------|--------------------------------------------|
| Offer ID        | CAX-OL-2026-290                            |
| Project Name    | Real-Time Driver Drowsiness Detection      |
| Author          | Muhammad Azeen Waqas                       |
| Institution     | COMSATS University Islamabad, Wah Campus   |
| Program         | BS Artificial Intelligence (Batch 2023–27) |
| Submission Date | 01-08-2026                                 |
| Document Type   | Project Report (PDF)                       |
| Version         | 1.0 — Final                                |

TABLE OF CONTENTS

---

|                                |    |
|--------------------------------|----|
| 1. Introduction.....           | 3  |
| 2. Problem Statement.....      | 3  |
| 3. Objectives.....             | 4  |
| 4. Dataset Description.....    | 4  |
| 5. Methodology.....            | 5  |
| 6. Algorithms Used.....        | 6  |
| 7. Experimental Results.....   | 8  |
| 8. Performance Evaluation..... | 9  |
| 9. Challenges Faced.....       | 10 |
| 10. Conclusion.....            | 11 |
| 11. Future Scope.....          | 11 |
| 12. References.....            | 12 |

## 1. Introduction

---

Driver drowsiness is a critical road safety issue that leads to thousands of preventable fatalities every year. Unlike distracted driving or speeding, drowsiness is an **internal physiological state** that is difficult for drivers themselves to recognize in time. Traditional safety mechanisms — road rumble strips, lane departure warning systems — are purely reactive and engage only after the vehicle has already drifted into danger.

This project, **DrowsyGuard**, addresses this gap by building a **proactive, real-time drowsiness detection system** that continuously monitors the driver's facial cues using a standard webcam and computer vision algorithms. The system provides timely, multi-modal alerts (audio and visual) before a safety incident can occur.

The project is implemented using **Python, OpenCV, MediaPipe FaceMesh, scikit-learn (SVM), FastAPI**, and **Streamlit**. It covers the complete AI/ML pipeline: data generation, feature engineering, model development, hyperparameter tuning, evaluation, and deployment.

## 2. Problem Statement

---

Fatigue and drowsiness are responsible for an estimated **20–30% of all road traffic accidents** globally (WHO, 2023). Studies indicate:

- Drowsy drivers have reaction times comparable to drunk drivers (BAC 0.08%)
- Microsleep episodes (involuntary sleep lasting 1–30 seconds) can occur without the driver's awareness
- Commercial truck and bus drivers are at highest risk due to long, monotonous driving schedules
- Existing OEM in-car solutions (e.g., Mercedes Attention Assist, Volvo DAMS) are expensive and proprietary

There is a clear need for an **affordable, open-source, hardware-agnostic solution** that works on commodity hardware (laptop/dashcam) and can be extended by researchers and developers. DrowsyGuard fulfills this need.

## 3. Objectives

---

- Detect driver drowsiness in real-time (< 500ms latency) using facial landmark analysis
- Implement Eye Aspect Ratio (EAR), Mouth Aspect Ratio (MAR), and Head Pose estimation algorithms
- Train a supervised ML classifier (SVM) on extracted features achieving  $\geq 90\%$  accuracy
- Classify driver states into three levels: Alert → Drowsy → Very Drowsy

- Design a two-tier alert escalation system (soft alert at 2s, hard alert at 4s)
- Deploy a live monitoring dashboard using Streamlit
- Log all drowsiness events for post-session review and analysis

## 4. Dataset Description

Two public benchmark datasets were referenced for this project, along with a synthetic feature dataset generated from the feature extraction pipeline:

| Dataset                   | Source                   | Size         | Classes                      | Usage                    |
|---------------------------|--------------------------|--------------|------------------------------|--------------------------|
| YawDD (Yawning Detection) | Kaggle                   | ~800 MB      | Yawning / Non-Yawning        | Video analysis reference |
| NTHU-DDD (Drowsy Driver)  | Kaggle                   | ~2.1 GB      | Alert / Drowsy / Sleepy      | Validation benchmark     |
| Synthetic EAR/MAR Dataset | Generated (this project) | 5000 samples | Alert / Drowsy / Very Drowsy | Model training           |

### 4.1 Synthetic Dataset Generation

The synthetic data was generated using statistical distributions derived from published benchmark studies:

| Class       | Label | Samples | EAR Range | MAR Range | PERCLOS Range |
|-------------|-------|---------|-----------|-----------|---------------|
| Alert       | 0     | 2,500   | 0.22–0.45 | 0.10–0.60 | 0.00–0.20     |
| Drowsy      | 1     | 1,500   | 0.12–0.26 | 0.35–0.85 | 0.15–0.60     |
| Very Drowsy | 2     | 1,000   | 0.05–0.20 | 0.60–1.20 | 0.40–0.95     |

Features: **EAR, MAR, head\_pitch, PERCLOS, ear\_mar\_ratio, pitch\_norm, drowsy\_score.**

## 5. Methodology

### 5.1 System Pipeline

| Layer | Component          | Technology            | Function                          |
|-------|--------------------|-----------------------|-----------------------------------|
| 1     | Video Capture      | OpenCV VideoCapture   | Acquire frames at 20–30 FPS       |
| 2     | Face Detection     | MediaPipe FaceMesh    | Locate face, 468 landmarks        |
| 3     | Feature Extraction | Custom Python (scipy) | Compute EAR, MAR, Head Pose       |
| 4     | State Classifier   | SVM (scikit-learn)    | Predict: Alert/Drowsy/Very Drowsy |
| 5     | Alert Engine       | pygame + OpenCV       | Audio beep + visual overlay       |

|   |              |                     |                               |
|---|--------------|---------------------|-------------------------------|
| 6 | API Server   | FastAPI + WebSocket | Stream state data to frontend |
| 7 | Dashboard    | Streamlit           | Live monitoring interface     |
| 8 | Event Logger | Python CSV writer   | Persist drowsiness events     |

5.2 Data Preprocessing

- Outlier removal using IQR clipping ( $EAR \in [0, 0.5]$ )
- Feature engineering: ear\_mar\_ratio, pitch\_norm, drowsy\_score composite feature
- Stratified 70/10/20 train/validation/test split
- StandardScaler normalization fitted only on training data

5.3 Alert Escalation Logic

| Time (s) | Condition                 | Alert Level     | Action                        |
|----------|---------------------------|-----------------|-------------------------------|
| 0        | EAR drops below 0.22      | Detection start | Frame counter begins          |
| 2        | EAR below threshold (2s)  | SOFT ALERT      | Beep sound (1000 Hz, 200ms)   |
| 4        | EAR below threshold (4s)  | HARD ALERT      | Loud alarm + red screen flash |
| Reset    | EAR recovers for 3 frames | ALERT restored  | Counter resets, alerts stop   |

6. Algorithms Used

6.1 Eye Aspect Ratio (EAR)

The EAR algorithm was introduced by Soukupová and ■ech (2016). It quantifies eye openness using six facial landmarks:

$$EAR = (||p2 - p6|| + ||p3 - p5||) / (2.0 \times ||p1 - p4||)$$

When both eyes are open,  $EAR \approx 0.25\text{--}0.35$ . A sustained EAR below **0.22** for more than 2 seconds indicates drowsiness.

6.2 Mouth Aspect Ratio (MAR)

MAR detects yawning using 12 mouth landmark coordinates:

$$MAR = (||p2-p8|| + ||p3-p7|| + ||p4-p6||) / (2.0 \times ||p1-p5||)$$

Normal:  $MAR < 0.5$ . Yawning:  $MAR > \mathbf{0.65}$ .

6.3 Head Pose Estimation

Head pitch is estimated using OpenCV's solvePnP with six 3D reference landmarks:

```
cv2.solvePnP(model_3d_points, image_2d_points, camera_matrix, dist_coeffs)
```

```
pitch = math.degrees(math.asin(-rotation_matrix[2][0]))
```

A pitch exceeding 20° combined with low EAR indicates microsleep.

6.4 PERCLOS

PERCLOS (Wierwille & Ellsworth, 1994) measures eye closure percentage over a rolling 60-frame window:

```
PERCLOS = count(EAR[i] < EAR_THRESHOLD) / 60 frames
```

- PERCLOS < 0.20 → Alert
- 0.20–0.50 → Drowsy
- > 0.50 → Very Drowsy

6.5 SVM Classifier

A **Support Vector Machine** with RBF kernel is selected because: the EAR/MAR feature space has non-linear class boundaries; SVMs are fast at inference (real-time critical); and they generalize well with limited data. Best hyperparameters (GridSearchCV, 5-fold CV): **C=10, gamma='scale', kernel='rbf'**.

7. Experimental Results

| Model                 | Accuracy | Precision | Recall | F1-Score | Train Time |
|-----------------------|----------|-----------|--------|----------|------------|
| Rule-Based (Baseline) | 85.3%    | 84.7%     | 85.3%  | 84.9%    | <1s        |
| SVM (RBF)             | 93.8%    | 93.5%     | 93.8%  | 93.6%    | 1.2s       |
| Random Forest         | 94.0%    | 93.8%     | 94.0%  | 93.8%    | 3.5s       |
| Gradient Boosting     | 94.4%    | 94.1%     | 94.4%  | 94.2%    | 8.7s       |
| SVM (Tuned, C=10)     | 94.2%    | 93.9%     | 94.2%  | 94.0%    | 2.1s       |

The **SVM (Tuned)** is selected for deployment — optimal balance of accuracy and inference speed.

7.1 Per-Class Performance — Best SVM

| Class       | Precision | Recall | F1-Score | Support |
|-------------|-----------|--------|----------|---------|
| Alert       | 95.1%     | 96.3%  | 95.7%    | 500     |
| Drowsy      | 93.4%     | 92.8%  | 93.1%    | 300     |
| Very Drowsy | 94.2%     | 93.5%  | 93.8%    | 200     |
| Macro Avg   | 94.2%     | 94.2%  | 94.2%    | 1000    |

7.2 AUC-ROC Scores

| Class         | AUC-ROC | Interpretation                                    |
|---------------|---------|---------------------------------------------------|
| Alert         | 0.989   | Excellent — very high separation from non-alert   |
| Drowsy        | 0.971   | Excellent — reliable detection of fatigue onset   |
| Very Drowsy   | 0.983   | Excellent — near-perfect critical state detection |
| Macro Average | 0.981   | Excellent overall discriminative ability          |

8. Performance Evaluation

| Metric              | Target  | Achieved | Status |
|---------------------|---------|----------|--------|
| Detection Accuracy  | ≥ 90%   | 94.2%    | ✓ PASS |
| AUC-ROC             | ≥ 0.95  | 0.981    | ✓ PASS |
| Inference Latency   | < 500ms | ~40ms    | ✓ PASS |
| FPS (i5 CPU)        | ≥ 20    | 24–28    | ✓ PASS |
| Alert Trigger Time  | < 2.5s  | 2.0s     | ✓ PASS |
| False Positive Rate | < 5%    | 3.8%     | ✓ PASS |
| Dashboard Load Time | < 3s    | ~1.5s    | ✓ PASS |

The system processes each frame in approximately **35–45ms** on a standard Intel Core i5 (8GB RAM, no GPU), achieving 24–28 FPS sustained.

9. Challenges Faced

• Occlusion & Glasses

MediaPipe landmark detection degrades with sunglasses. Mitigated with a face confidence threshold and head-pose-only fallback.

• Variable Lighting

EAR values shift under low-light. Addressed with CLAHE preprocessing.

• False Positives from Natural Blinks

Normal blinks triggered alerts. Solved by requiring EAR below threshold for ≥ 48 consecutive frames (2 seconds at 24 FPS).

- **Head Pose Instability**

solvePnP was noisy on fast movement. Applied EMA:  $\text{pitch\_smooth} = 0.7 \times \text{pitch\_prev} + 0.3 \times \text{pitch\_raw}$ .

- **Real-Time Dashboard Latency**

WebSocket lag at high resolution. Fixed by compressing JPEG to 70% and resizing to 640×480.

- **Class Imbalance**

Very Drowsy class underrepresented. Addressed with stratified split and `class_weight='balanced'`.

## 10. Conclusion

---

DrowsyGuard successfully demonstrates that **real-time driver drowsiness detection is achievable with commodity hardware and open-source tools**. By combining EAR, MAR, and head pitch with a tuned SVM, the system achieves **94.2% accuracy** with sub-50ms latency.

The modular architecture allows independent improvement of any layer. The Streamlit deployment makes it immediately accessible for research and demonstration. The project fulfills all CoderAxo internship requirements across the complete AI/ML pipeline.

## 11. Future Scope

---

- BiLSTM Temporal Model: Process 60-frame EAR sequences for temporal pattern detection
- Night Vision: IR/thermal camera preprocessing for nighttime reliability
- YOLOv8-Face: Superior side-face and occlusion handling
- Mobile App (Flutter + TFLite): On-device dashcam integration
- Physiological Fusion: Combine EAR with remote PPG heart rate
- Fleet Monitoring: Scale to 8+ streams with Redis + cloud dashboard
- Driver Identification: Personalize EAR thresholds per driver via face recognition

## 12. References

---

- [1] Soukupová, T., & Mach, J. (2016). Real-time eye blink detection using facial landmarks. 21st Computer Vision Winter Workshop.



- [2] Wierwille, W. W., & Ellsworth, L. A. (1994). Evaluation of driver drowsiness by trained raters. *Accident Analysis & Prevention*, 26(5), 571–581.
- [3] Lugaresi, C., et al. (2019). MediaPipe: A framework for perceiving and processing reality. *CVPR Workshop on CV for AR/VR*.
- [4] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- [5] Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
- [6] YawDD Dataset: <https://www.kaggle.com/datasets/davidvazquezcic/yawn-dataset>
- [7] NTHU-DDD Dataset: <https://www.kaggle.com/datasets/rakibuleceruet/drowsiness-prediction-dataset>
- [8] WHO Road Traffic Injuries Fact Sheet (2023). World Health Organization.
- [9] FastAPI Documentation: <https://fastapi.tiangolo.com>
- [10] Streamlit Documentation: <https://docs.streamlit.io>